
Table of Contents

.....	1
Résolution d'un problème inverse: déconvolution d'un signal triangulaire	1
1. Construction du jeu de données	1
a. Simulation du signal d'entrée (Question 1)	1
b. Simulation du filtre (Question 2)	2
c. Simulation du problème direct: première approche (Question 5 à 7)	3
d. Simulation du problème direct par approche matricielle (Question 8 à 10)	6
2. Déconvolution	7
a. Déconvolution « naïve » et constat de l'instabilité (Question 11 à 13)	7
b. Déconvolution par moindres carrés (Question 14 à 16)	12
c. Déconvolution par régularisation linéaire quadratique (Question 18 à 20)	16

```
clc;
clear all;
close all;
```

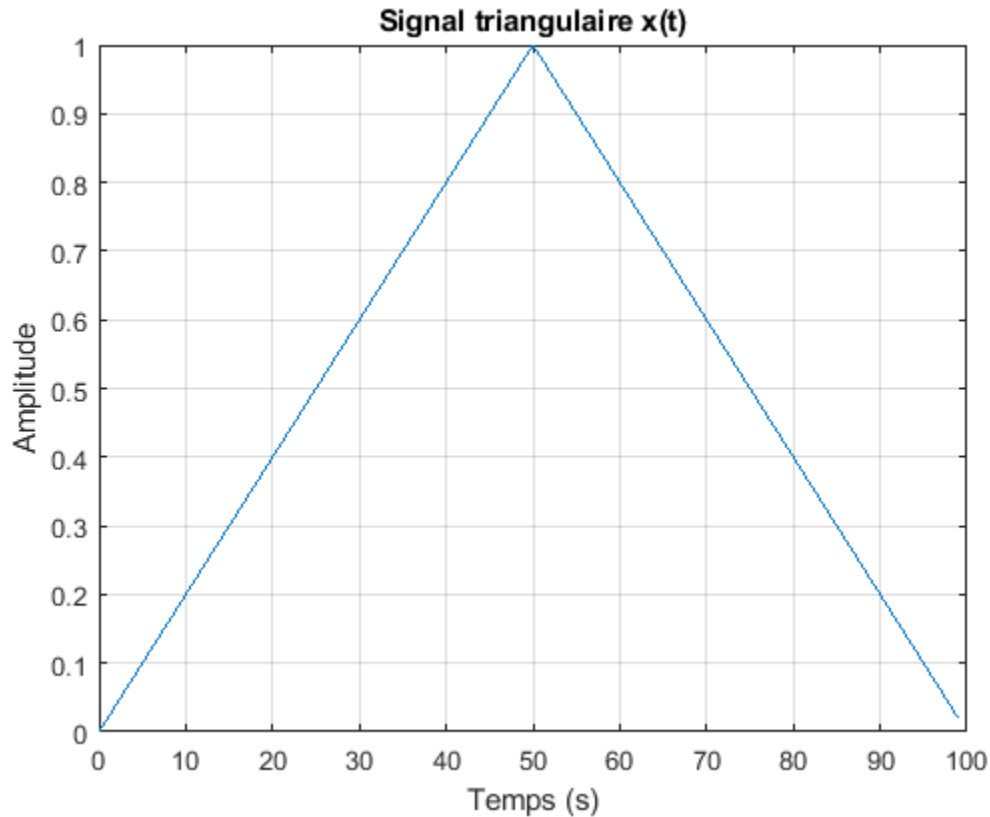
Résolution d'un problème inverse: déconvolution d'un signal triangulaire

1. Construction du jeu de données

```
% Paramètres
L = 100;           % Durée d' observation du signal d' entree ( secondes )
fe=1;              % Frequence d' echantillonnage (Hz)
Te = 1/ fe ;       % Periode d' echantillonnage ( secondes )
Nx = L/ Te ;       % Nombre d' echantillons du signal x
nx = 0: Nx -1;     % Indice temporel ( entier )
tx = nx * Te ;     % Base de temps ( secondes )
```

a. Simulation du signal d'entrée (Question 1)

```
x1 = (tx(1:Nx/2)) / (L/2); % Première partie  $\forall t \in [0, L/2]$  (croissante)
x2 = (-tx(Nx/2+1:end) + L) / (L/2); % Deuxième partie  $\forall t \in [L/2, L]$ 
(décroissante)
x = [x1, x2]; % Signal triangulaire complet
% Affichage du signal d'entrée
figure;
plot(tx, x); title('Signal triangulaire x(t)');
xlabel('Temps (s)'); ylabel('Amplitude'); grid on;
```

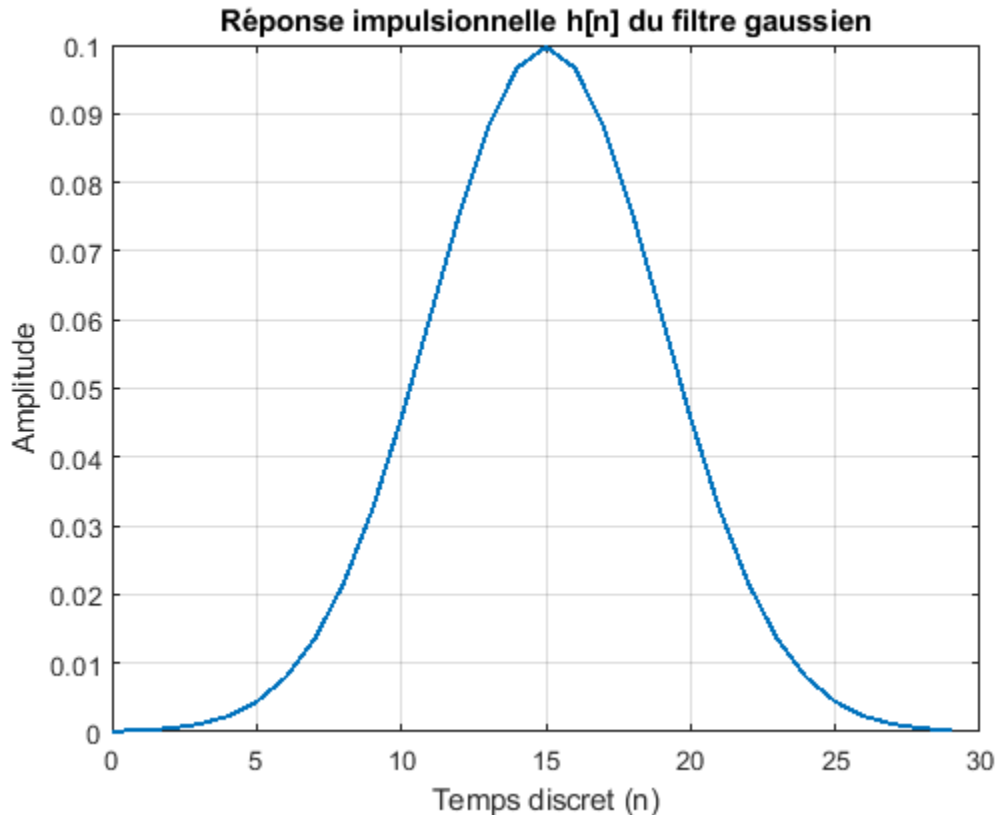


b. Simulation du filtre (Question 2)

```
% Paramètres du filtre
m = 15;           % Moyenne
sigma = 4;        % Écart-type
Nh = 30;          % Horizon temporel
th = (0:Nh-1)*Te; % Base de temps du filtre

% Réponse impulsionnelle
h = (1 / (sigma * sqrt(2 * pi))) * exp(-((th - m).^2) / (2 * sigma^2));

% Affichage de la réponse impulsionnelle
figure;
plot(th, h, 'LineWidth', 1.5);
xlabel('Temps discret (n)'); ylabel('Amplitude');
title('Réponse impulsionnelle h[n] du filtre gaussien');
grid on;
```

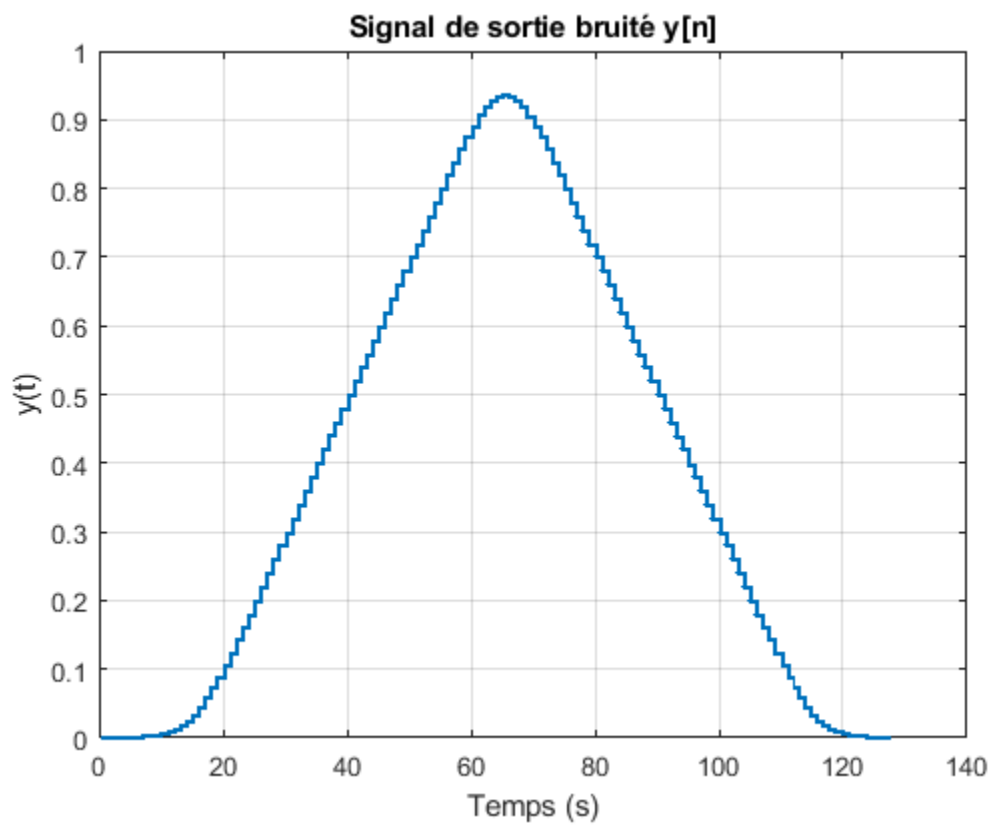
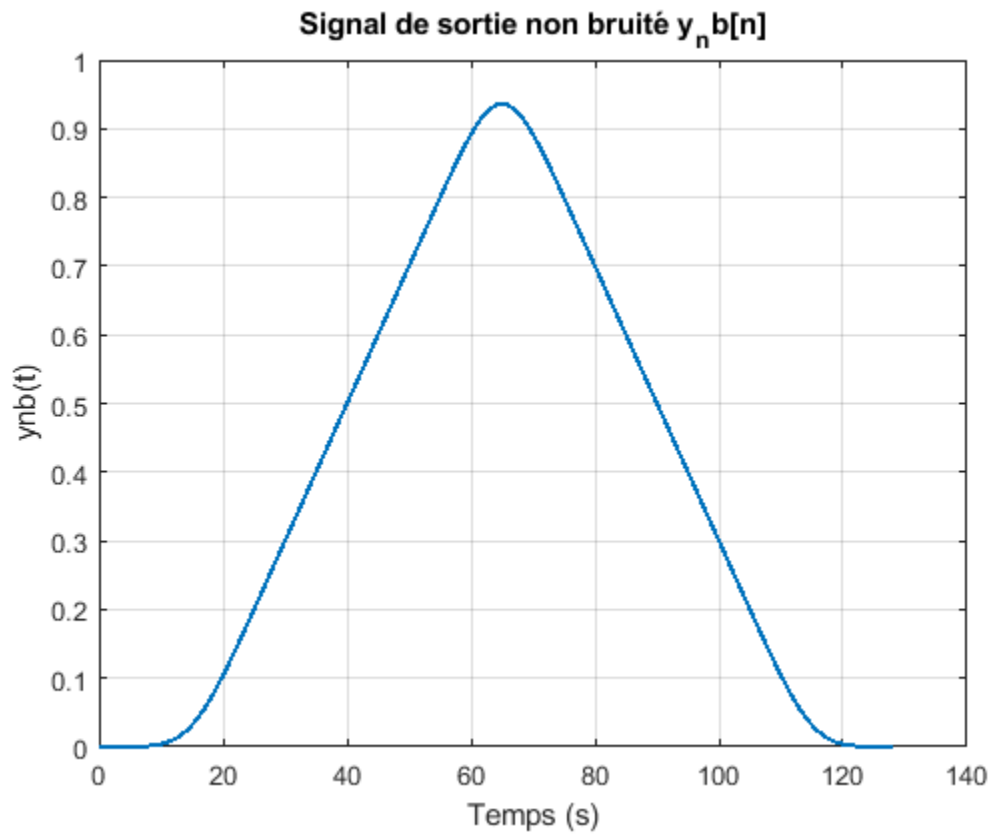


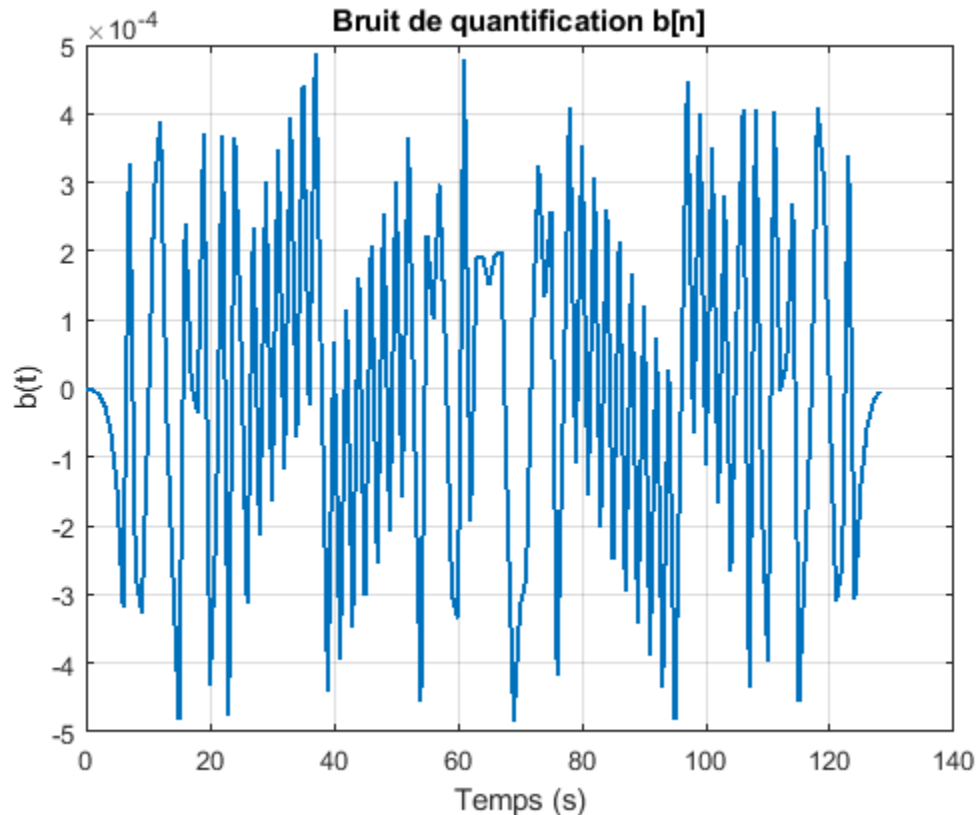
c. Simulation du problème direct: première approche (Question 5 à 7)

```
% Produit de convolution discret
ynb = conv(x, h, 'full');           % Convolution complète
tnb = (0:length(ynb)-1) * Te;      % Base de temps associée
% Quantification du signal
y = round(ynb * 2^10) / 2^10;
% Calcul du bruit de quantification
b = y - ynb;                       %bruit de quantification
figure;
% Signal de sortie non bruité
figure;
plot(tnb, ynb, 'LineWidth', 1.5);
xlabel('Temps (s)');ylabel('ynb(t)');
title('Signal de sortie non bruité y_nb[n]');
grid on;

% Signal de sortie bruité
figure;
stairs(tnb, y, 'LineWidth', 1.5); % stairs pour un affichage "discret"
xlabel('Temps (s)');ylabel('y(t)');
title('Signal de sortie bruité y[n]');
grid on;
```

```
% Bruit de quantification
figure;
plot(tnb, b, 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('b(t)');
title('Bruit de quantification b[n]');
grid on;
```





d. Simulation du problème direct par approche matricielle (Question 8 à 10)

```
Nh = length(h); % Nombre d'échantillons de la réponse impulsionnelle h[n]
Nx = length(x); % Nombre d'échantillons du signal d'entrée x[n]
Ny = Nx + Nh - 1; % Taille complète de la sortie ynb

% Initialisation de la première colonne et de la première ligne de la
matrice Toeplitz
CH = [h(:); zeros(Nx - 1, 1)]; % Première colonne : h suivi de zéros
LH = [h(1), zeros(1, Nx - 1)]; % Première ligne : h(1) suivi de zéros

% Construction de la matrice Toeplitz H
H = toeplitz(CH, LH);

% Calcul de la sortie non bruitée par multiplication matricielle
ynb_mat = H * x(:); % Produit matriciel H * x

% Générer la base de temps pour la sortie complète
tnb_full = (0:Ny-1) * Te;

% Affichage du signal obtenu par l'approche matricielle
figure;
plot(tnb_full, ynb, 'r', 'LineWidth', 1.5); hold on;
```

```

plot(tnb_full, ynb_mat, 'b--', 'LineWidth', 1.5);
legend('Convolution classique (conv)', 'Multiplication matricielle (H*x)');
xlabel('Temps discret (n)'); ylabel('Amplitude');
title('Comparaison entre la convolution et l\'approche matricielle sans
troncature');
grid on;

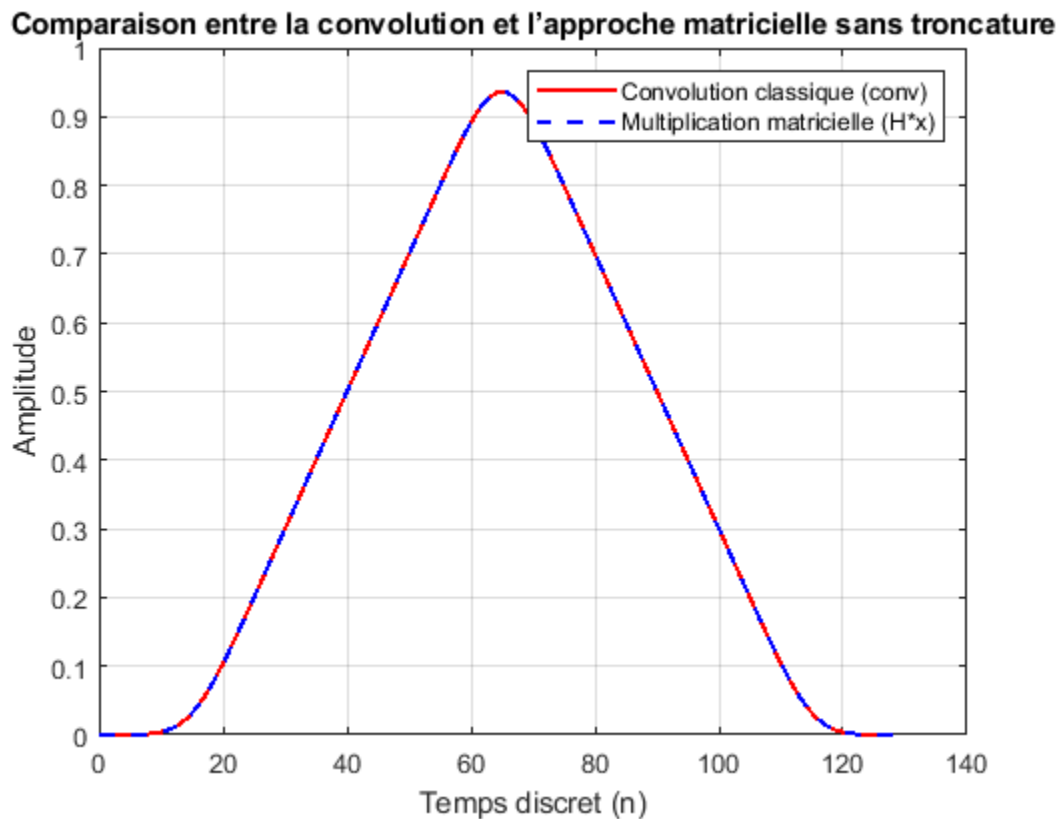
```

```

% Vérification de la différence entre les deux approches
err_co = norm(ynb - ynb_mat); % Erreur convolution
disp(['Erreur entre convolution et approche matricielle : ',
num2str(err_co)]);

```

Erreur entre convolution et approche matricielle : 41.5351



2. Déconvolution

a. Déconvolution « naïve » et constat de l'instabilité (Question 11 à 13)

Déconvolution avec signal non bruité

```

x_rec_nb = deconv(ynb, h);

% Générer la base de temps pour x_rec_nb

```

```

t_rec = (0:length(x_rec_nb)-1) * Te;

% Affichage du signal reconstruit sans bruit
figure;
plot(t_rec, x_rec_nb, 'r', 'LineWidth', 1.5);
xlabel('Temps (s)'); ylabel('Amplitude');
title('Signal reconstruit par déconvolution naïve (non bruité)');
grid on;

% Déconvolution avec le signal bruité y
x_rec_b = deconv(y, h);

% Générer la base de temps pour x_rec_b
t_rec_b = (0:length(x_rec_b)-1) * Te;

% Affichage du signal reconstruit bruité
figure;
plot(t_rec_b, x_rec_b, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)'); ylabel('Amplitude');
title('Signal reconstruit par déconvolution naïve (bruité)');
grid on;

% Tester plusieurs valeurs de sigma
sigma_values = [1, 5, 10, 15]; % Différentes largeurs du filtre gaussien

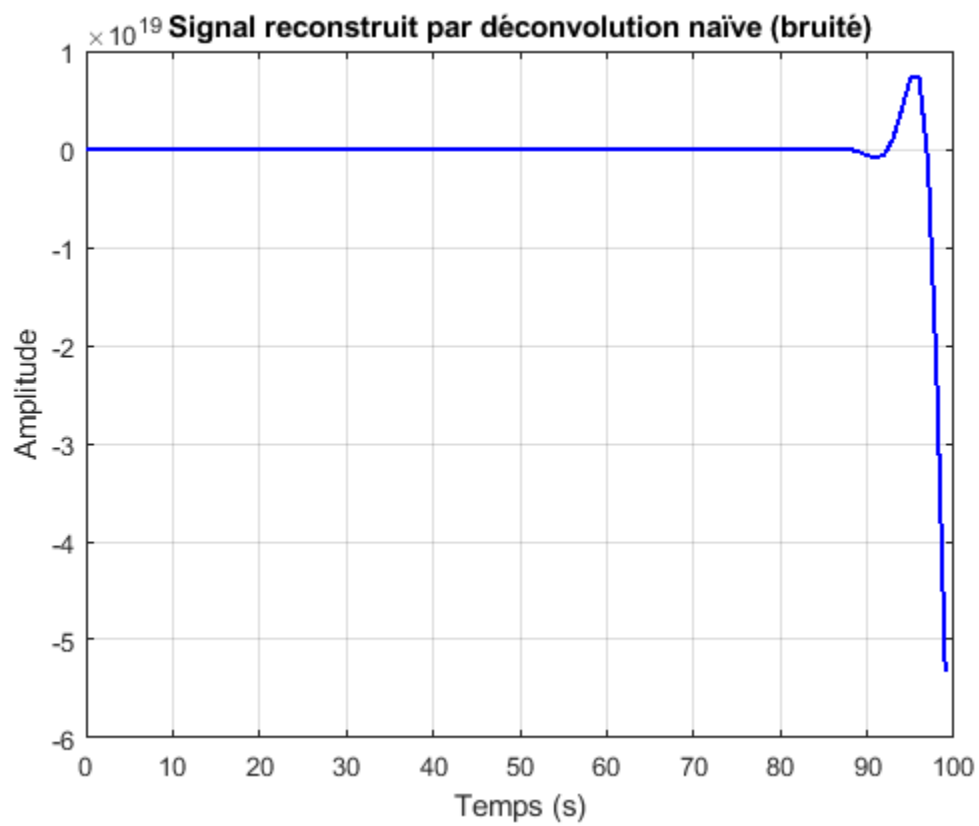
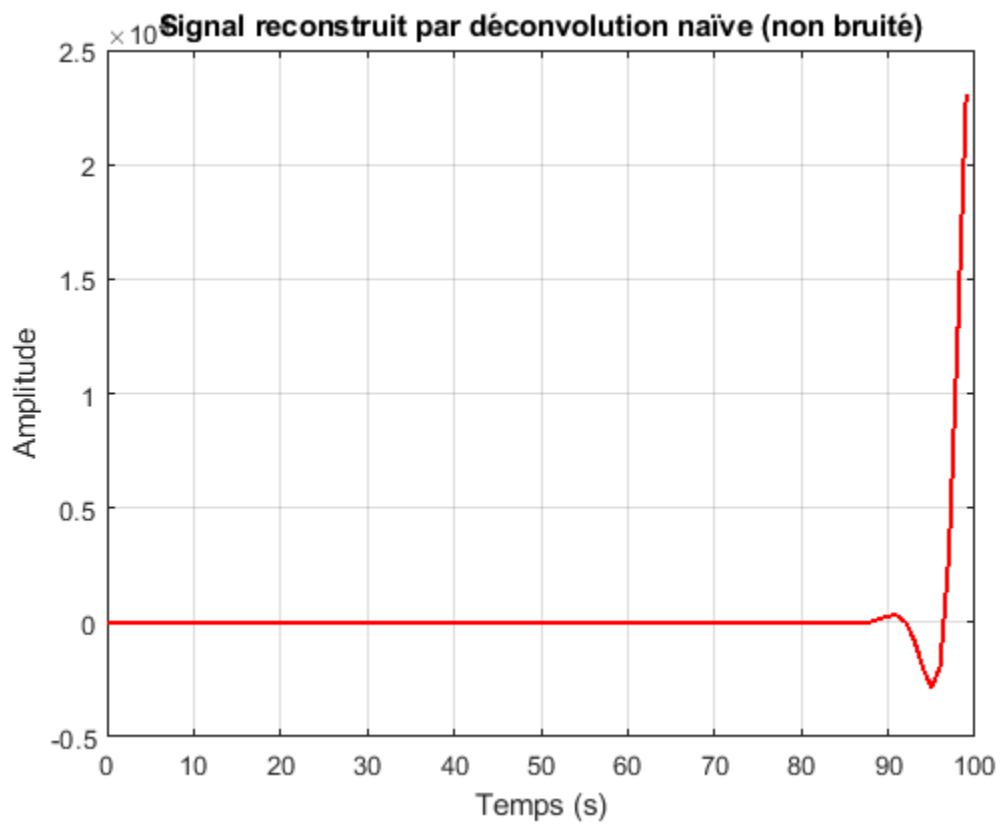
% Parcourir les différentes valeurs de sigma
for i=1:length(sigma_values)
    sigma_test = sigma_values(i);
    % Calculer la nouvelle réponse impulsionnelle avec le sigma modifié
    h_test = (1 / (sigma_test * sqrt(2 * pi))) * exp(-(th - m).^2) / (2 *
sigma_test^2));

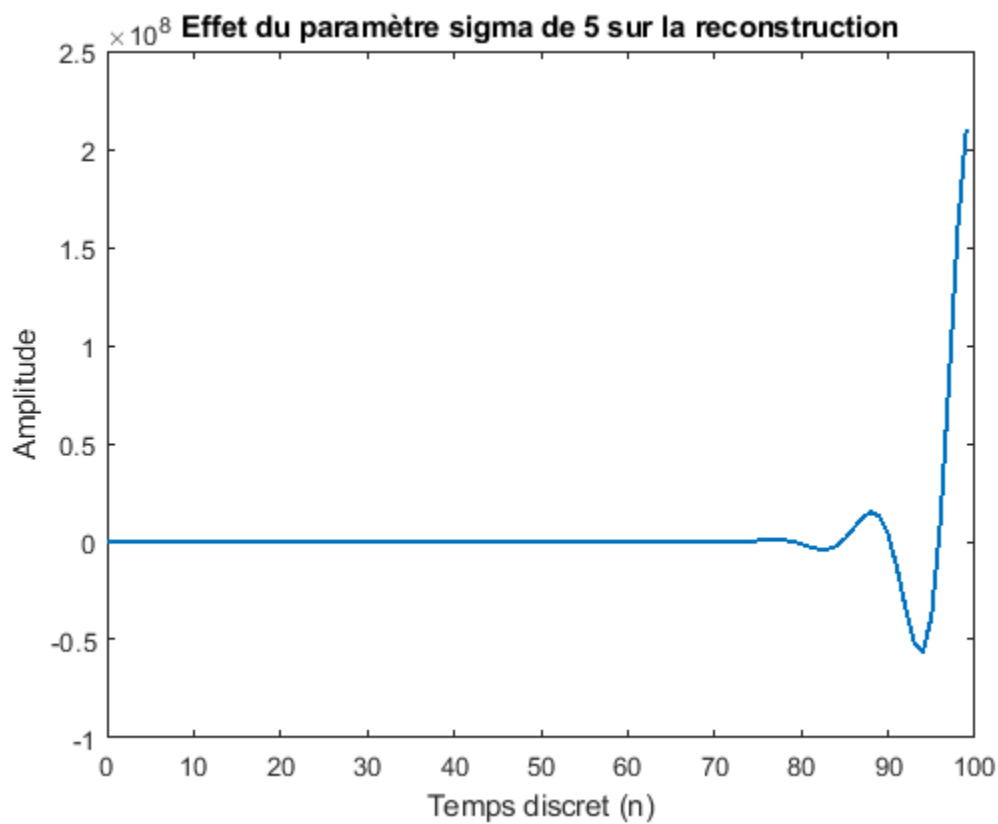
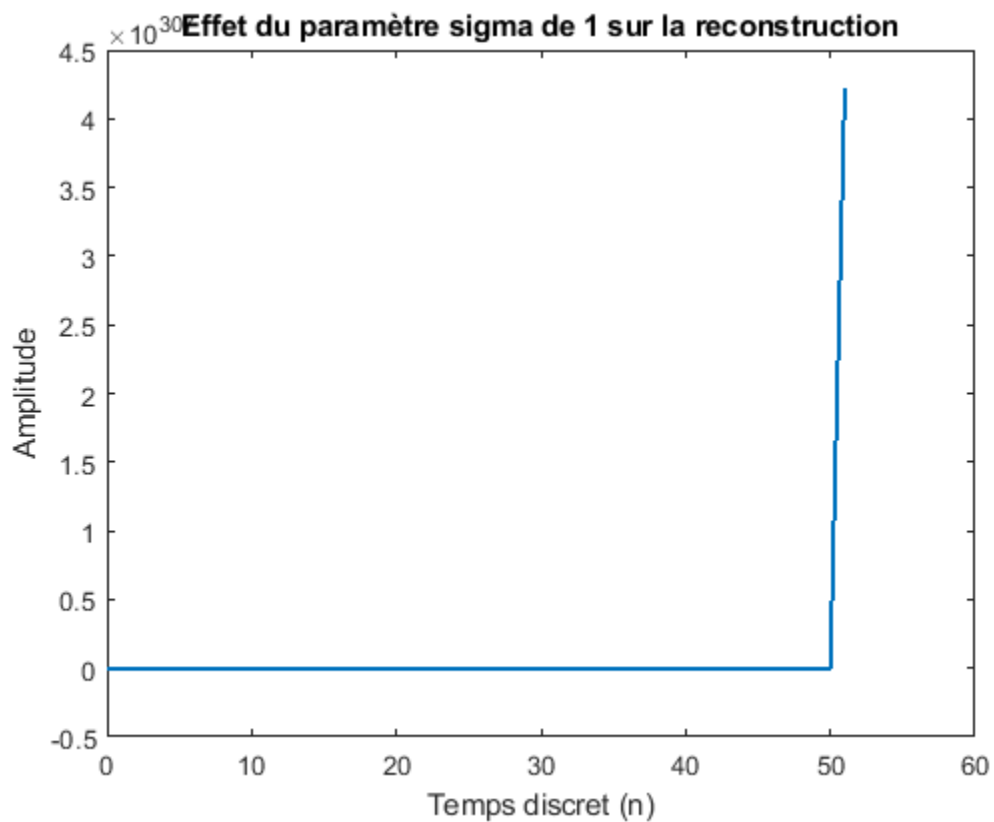
    % Déconvolution avec le nouveau filtre
    x_rec_t = deconv(y, h_test);

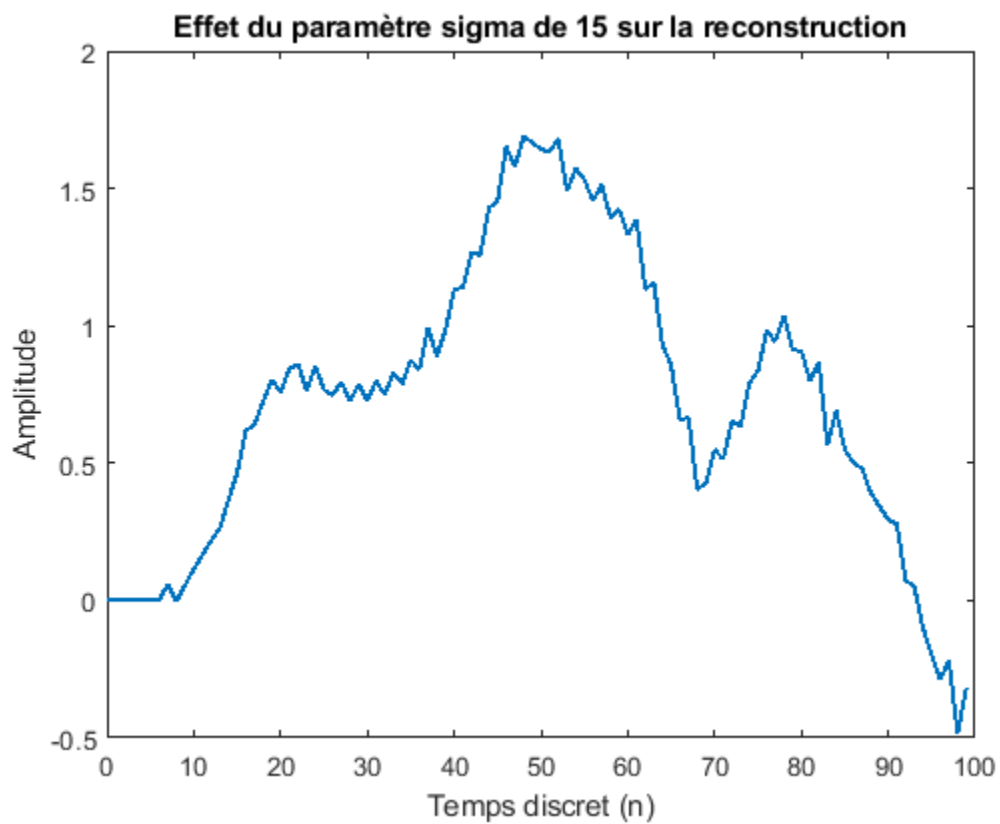
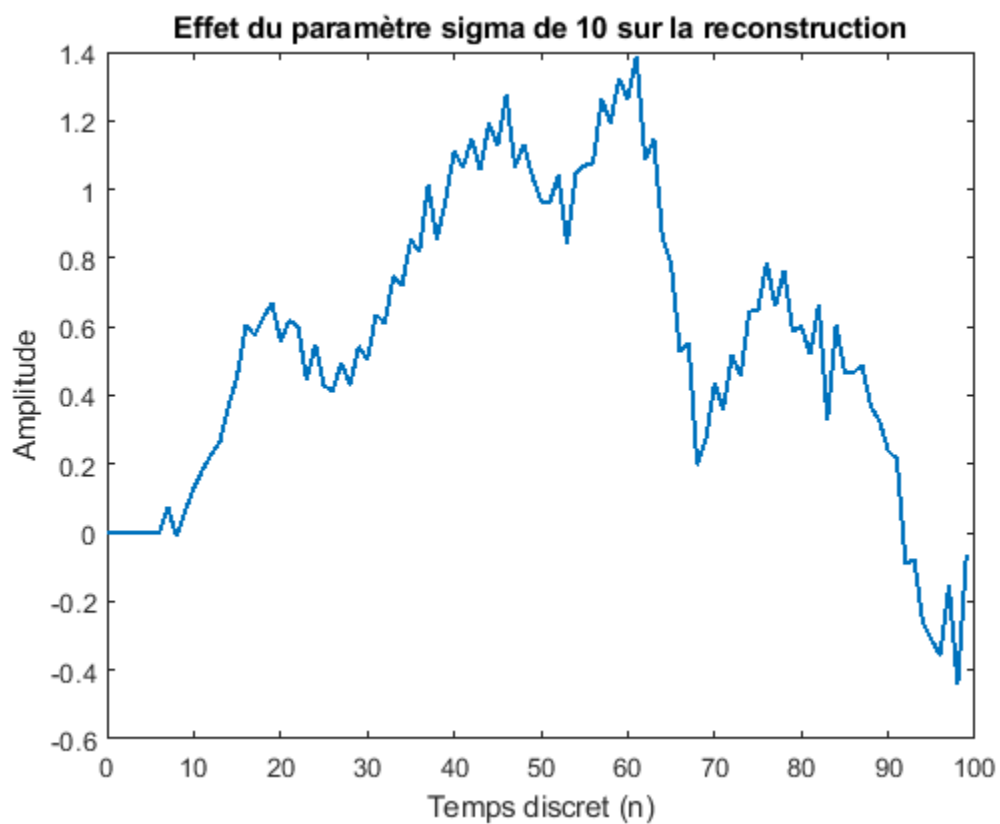
    % Générer la base de temps pour l'affichage
    t_rec_t = (0:length(x_rec_t)-1) * Te;

    % Tracer le signal reconstruit;
    figure; plot(t_rec_t, x_rec_t, 'LineWidth', 1.5, 'DisplayName', ['\sigma
= ', num2str(sigma_test)]);
    xlabel('Temps discret (n)'); ylabel('Amplitude');
    title(sprintf('Effet du paramètre sigma de %d sur la reconstruction',
sigma_test));
end

```







b. Déconvolution par moindres carrés(Question 14 à 16)

```
% Calcul du signal déconvolué par moindres carrés
x_lsqr = pinv(H) * y(:); % Inversion pseudo-inverse

% Affichage du signal reconstruit
figure;
plot(tx, x_lsqr, 'g', 'LineWidth', 1.5);
xlabel('Temps discret (n)'); ylabel('Amplitude');
title('Signal déconvolué par moindres carrés');
grid on;

% Energie de l'erreur de reconstruction
en_er = norm(x - x_lsqr); % Norme de la différence entre le signal original
et reconstruit
disp(['Erreur de reconstruction par moindres carrés : ', num2str(en_er)]);

% Décomposition en valeurs singulières (SVD)
singular_values = svd(H);
condition_number = singular_values(1) / singular_values(end);

% Affichage des valeurs singulières
figure;
semilogy(singular_values, 'o-', 'LineWidth', 1.5);
xlabel('Indice'); ylabel('Valeur singulière');
title('Valeurs singulières de H');
grid on;

disp(['Conditionnement de H : ', num2str(condition_number)]);

% Analyse du conditionnement en fonction de sigma (Question 17)
% Mise a jour de sigma
sigma_values = [1, 5, 10, 15, 20, 25];
condition_numbers = zeros(size(sigma_values));

% Parcourir les différentes valeurs de sigma
for i = 1:length(sigma_values)
    sigma_test = sigma_values(i);

    % Calculer la nouvelle réponse impulsionnelle avec le sigma modifié
    h_test = (1 / (sigma_test * sqrt(2 * pi))) * exp(-(th - m).^2) / (2 *
sigma_test^2));

    % Construction de la matrice Toeplitz associée
    CH_test = [h_test(:); zeros(Nx - 1, 1)];
    LH_test = [h_test(1), zeros(1, Nx - 1)];
    H_test = toeplitz(CH_test, LH_test);

    % Décomposition en valeurs singulières
    singular_values_test = svd(H_test);
```

```

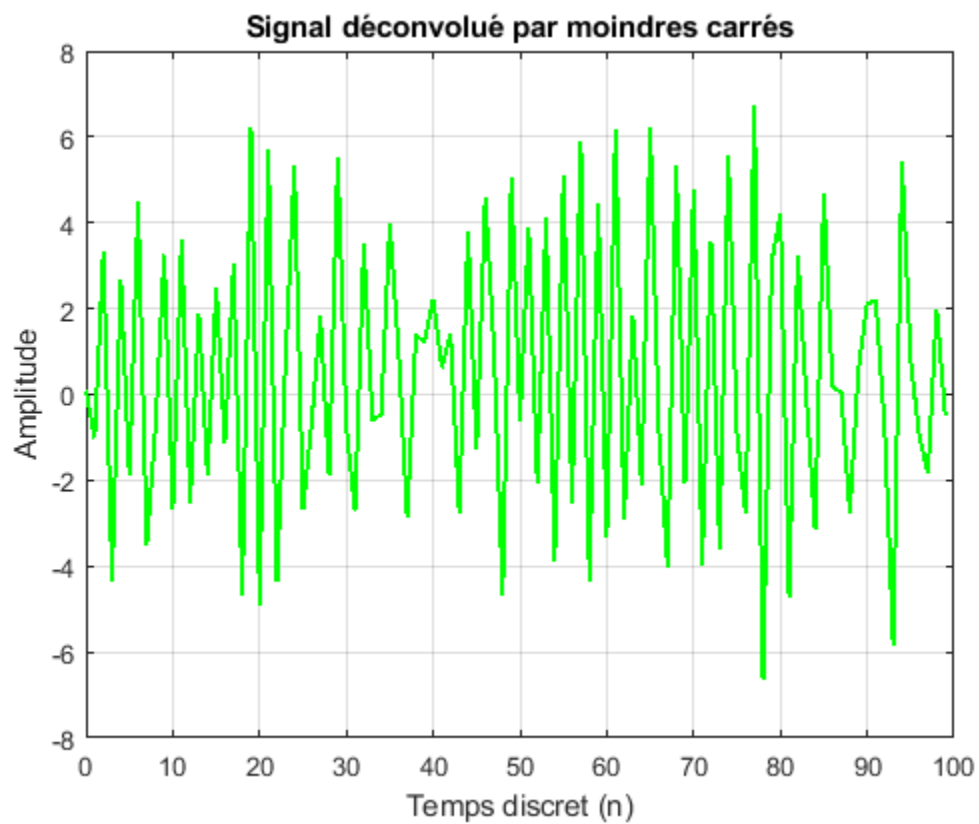
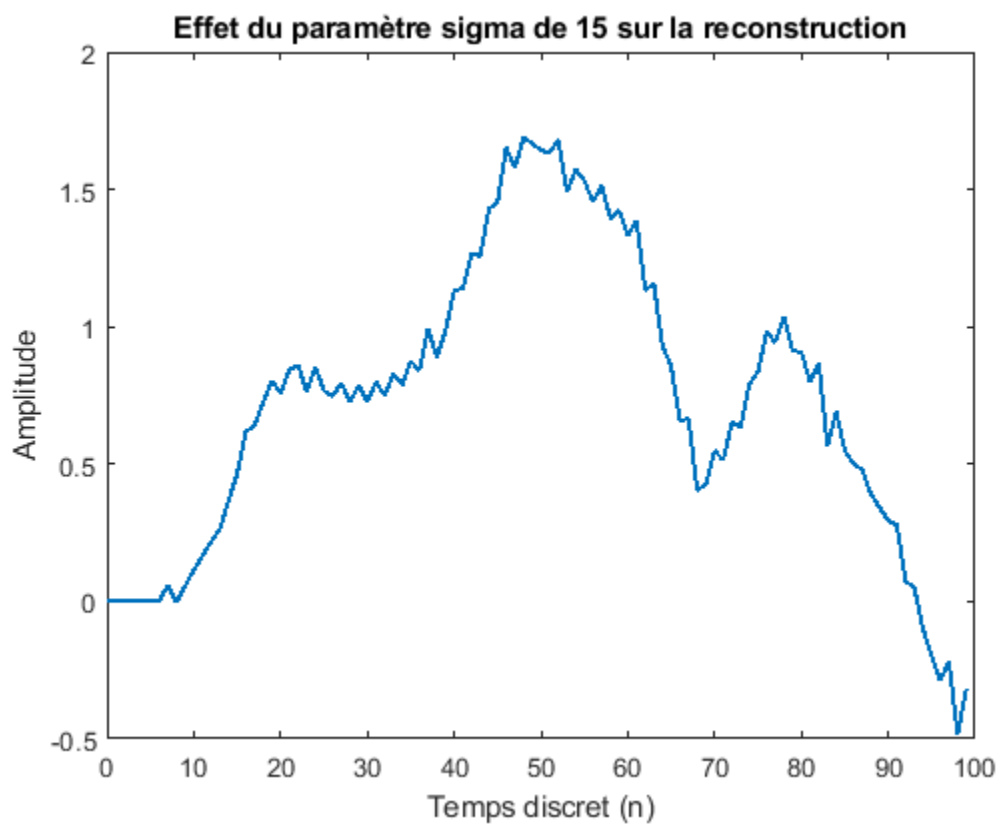
    % Vérification pour éviter la division par zéro
    if singular_values_test(end) ~= 0
        % Calcul du conditionnement
        condition_numbers(i) = singular_values_test(1) /
singular_values_test(end);
    else
        % Affichage d'un message d'avertissement
        fprintf('Attention : singular_values_test(end) = 0 . Conditionnement
non calculé.\n');
    end
end

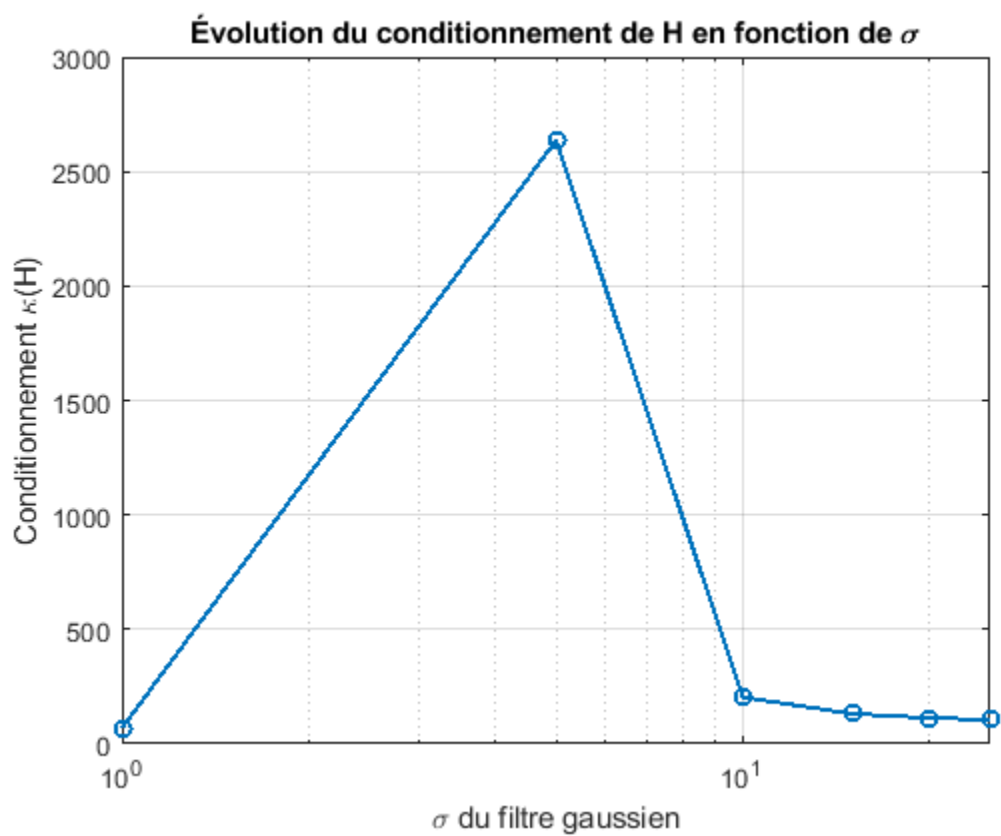
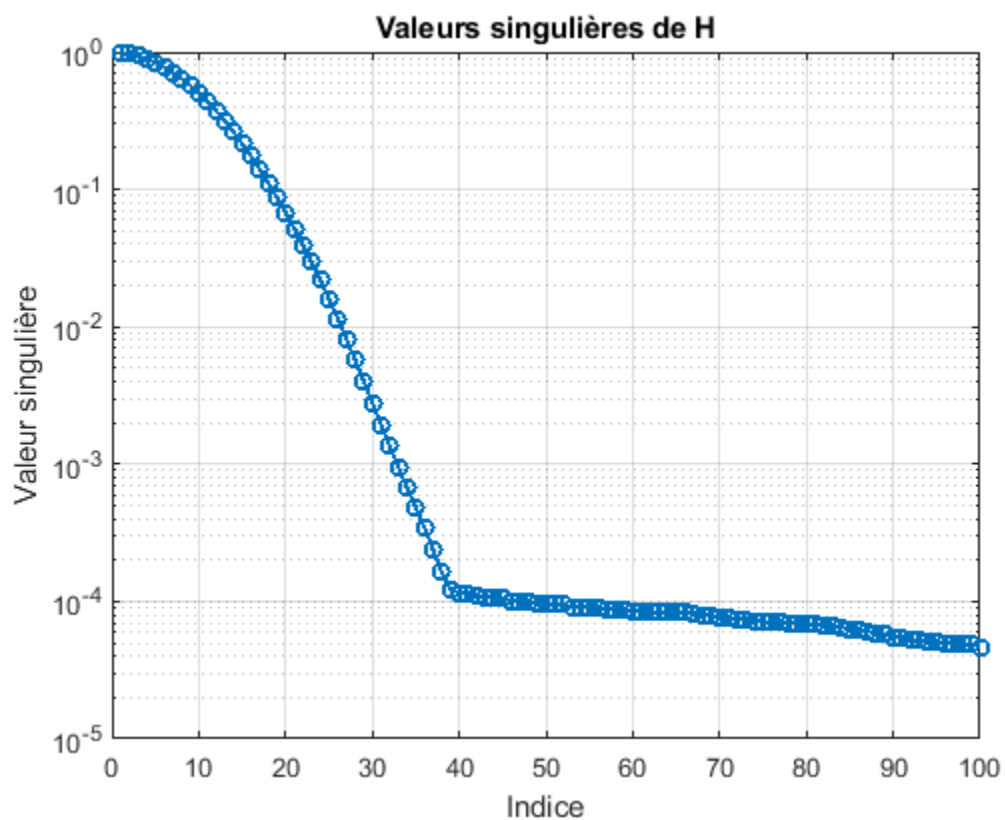
% Affichage du conditionnement en fonction de sigma
figure;
semilogx(sigma_values, condition_numbers, 'o-', 'LineWidth', 1.5);
xlabel('\sigma du filtre gaussien'); ylabel('Conditionnement \kappa(H)');
title('Évolution du conditionnement de H en fonction de \sigma');
grid on;

% Affichage du meilleur et pire conditionnement
disp(['Meilleur conditionnement (plus stable) pour sigma = ',
num2str(sigma_values(find(condition_numbers == min(condition_numbers), 1))),
...
', avec \kappa(H) = ', num2str(min(condition_numbers))]);
disp(['Pire conditionnement (plus instable) pour sigma = ',
num2str(sigma_values(find(condition_numbers == max(condition_numbers), 1))),
...
', avec \kappa(H) = ', num2str(max(condition_numbers))]);

Erreur de reconstruction par moindres carrés : 33.9559
Conditionnement de H : 21225.2935
Meilleur conditionnement (plus stable) pour sigma = 1, avec \kappa(H) =
69.1713
Pire conditionnement (plus instable) pour sigma = 5, avec \kappa(H) =
2636.0185

```





c. Déconvolution par régularisation linéaire quadratique(Question 18 à 20)

```
% Construction de la matrice D1
dcol1 = zeros(1,Nx);
dcol1 (1:2) = dcol1 (1:2) + [1 -1];
dlig1 = zeros (1 , Nx );
dlig1 (1 ,1) = dcol1 (1 ,1) ;
D1 = toeplitz ( dcol1 , dlig1 );

% Tester plage de valeurs pour lambda
lambda_values = logspace(-7, 2);
errors = zeros(size(lambda_values));

for i = 1:length(lambda_values)
    lambda = lambda_values(i);
    x_reg = (H' * H + lambda * D1' * D1) \ (H' * y(:)); % Reconstruction
    régulière
    errors(i) = norm(x - x_reg'); % Erreur de reconstruction
end

figure;
loglog(lambda_values, errors, '-o');
xlabel('\lambda');ylabel('Erreur de reconstruction');
title('Évolution de l'erreur de reconstruction en fonction de \lambda');
grid on;

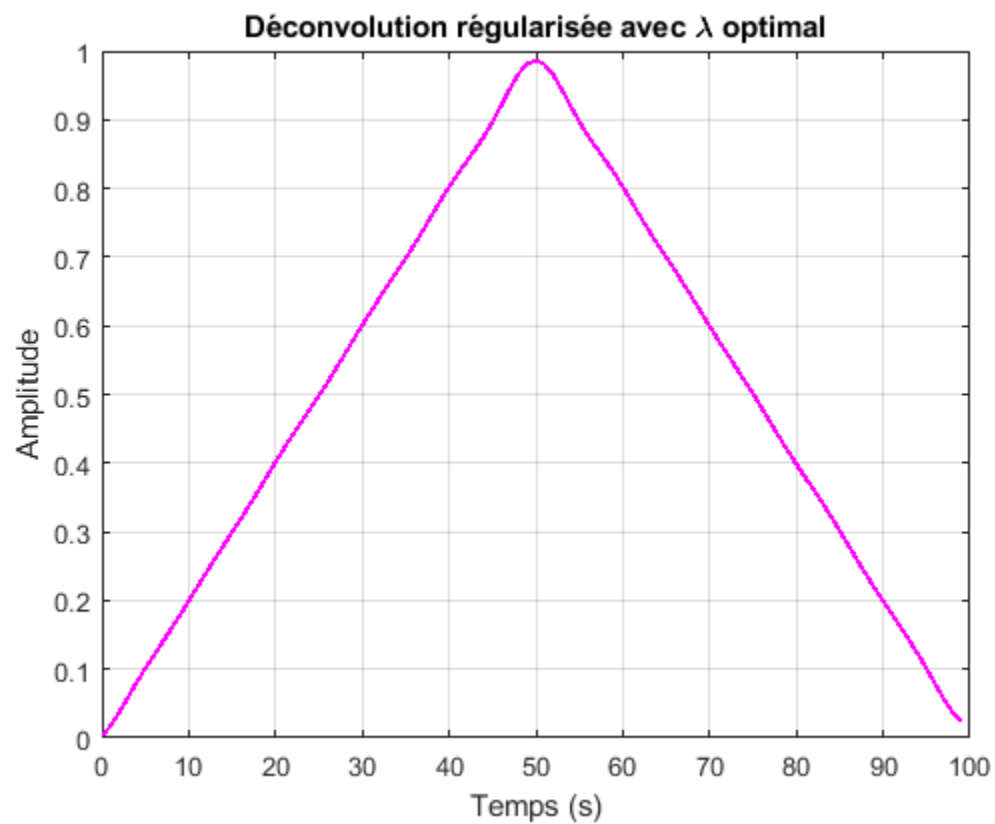
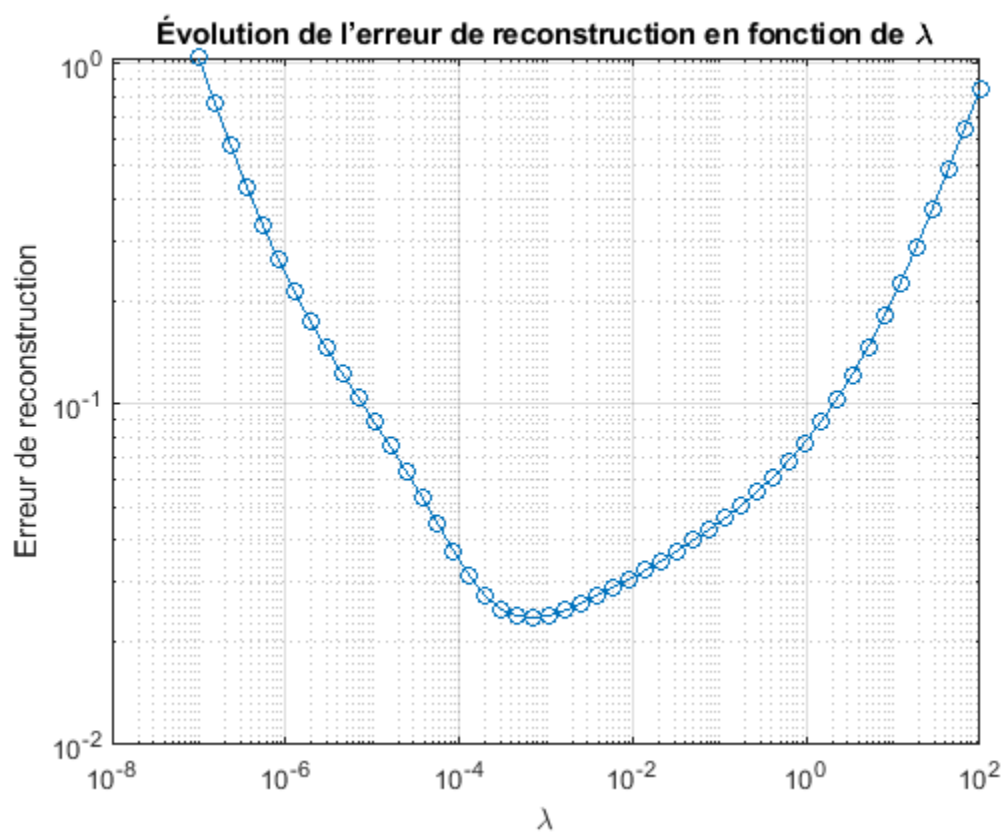
% Trouver la meilleure valeur de lambda
[~, idx] = min(errors); % Trouver l'indice de l'erreur minimale
lambda_opt = lambda_values(idx);

% Calcul de la solution régularisée
x_reg_opt = (H' * H + lambda_opt * D1' * D1) \ (H' * y(:));

% Affichage du signal reconstruit
figure;
plot(tx, x_reg_opt, 'm', 'LineWidth', 1.5);
xlabel('Temps (s)');ylabel('Amplitude');
title('Déconvolution régularisée avec \lambda optimal');
grid on;

disp(['Valeur optimale de \lambda : ', num2str(lambda_opt)]);

Valeur optimale de \lambda : 0.00071969
```

Published with MATLAB® R2024a